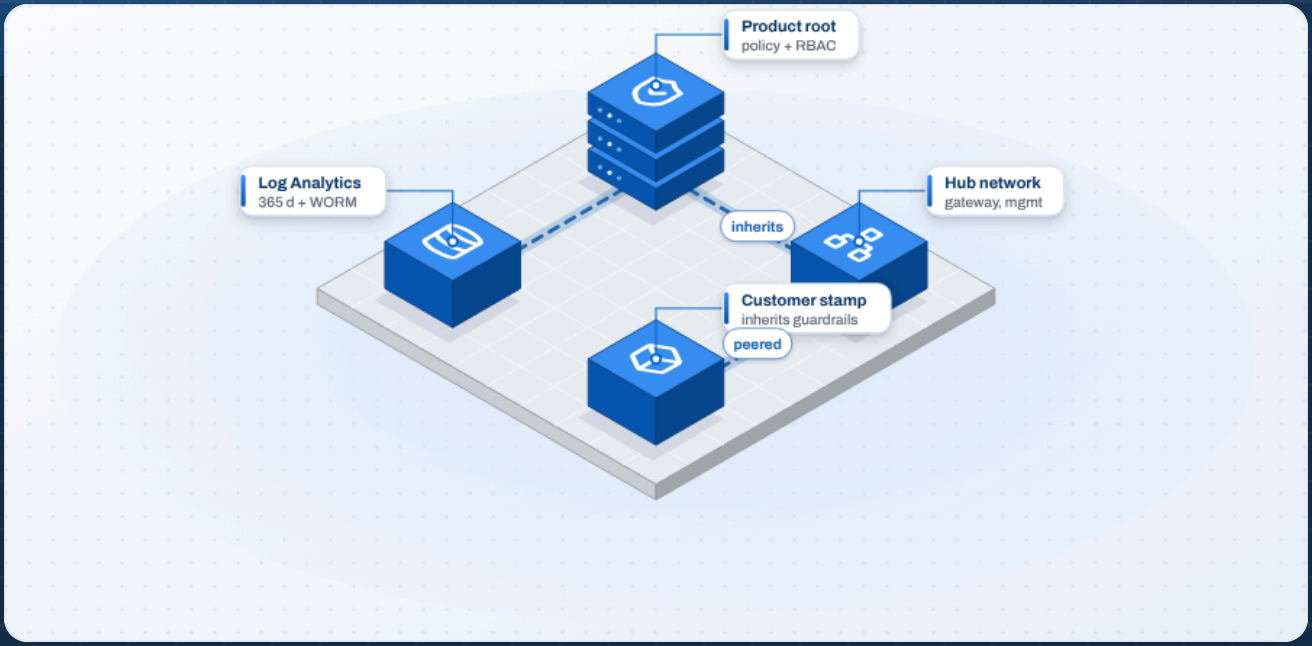
STANDARD OPERATING PROCEDURE · FIELD GUIDE SERIES

SOP-AZ-101

Building an Azure Landing Zone as Code

Management groups, group-based RBAC, Azure Policy guardrails, central logging with an immutable archive and the hub network, built once from a template and inherited by every subscription.

Cloud Infrastructure | Azure Policy | Landing Zone



DOCUMENT ID	SOP-AZ-101
VERSION	1.1
EFFECTIVE	October 2026
AUDIENCE	Platform engineers, cloud administrators, auditors (Chapter 5)
AUTHOR	Richmond Dieujuste, Principal Consultant

Rich Dieu IT Solutions LLC · richdieu.com

Sanitized reference edition · client identifiers removed

Contents

1. Concepts You Need First **2**

1.1 Management groups and inheritance 2

1.2 Guardrails: Deny, Modify and Audit 3

1.3 Logging: a working window and a record 4

2. Access, Prerequisites and Planning **5**

2.1 Level of access required 5

2.2 Prerequisites checklist 5

2.3 Decisions to make up front 6

3. Procedure in the Azure Portal **7**

4. Template · Foundation as Code **11**

5. Verification and Troubleshooting **14**

5.1 Verify the build 14

5.2 Troubleshooting 14

5.3 Rollback 14

6. Summary, Glossary and Review **15**

6.1 Chapter summary 15

6.2 Glossary 15

6.3 Framework references 15

LEARNING OBJECTIVES

After working through this procedure you will be able to:

- Explain how management groups, Azure Policy and RBAC inheritance shape everything built later.
- Build the foundation in the portal and from the PowerShell template, and know which is the version of record.
- Prove the exit criterion: every subscription under the product root and the deny guardrails at 100 percent compliance.

CHAPTER 1

Concepts You Need First

A landing zone is the part of a tenant that exists before any workload does: the folder structure for subscriptions, the rules every resource must follow, who may change what, and where every log goes. Everything a team deploys later inherits those decisions. Get them right once and every new customer or application starts compliant; get them wrong and each workload carries its own fix.

> **KEY CONCEPT · A BUILDING CODE, NOT A BUILDING**
>
> Think of the landing zone as the building code and the lot it sits on. It does not build the house (the workload); it decides where houses may go, what materials are banned, who holds the keys and which cameras record the street. A builder who follows the code passes inspection without extra work.

1.1 Management groups and inheritance

A **management group** is a container for subscriptions. Policy assignments and role assignments made on a management group flow down to every subscription and resource beneath it. This SOP builds one product root with four children: **Platform** (shared hub, logging, alerting), **Landing Zones** with **Customers** below it (one subscription per customer stamp), and **Sandbox** (no production data).

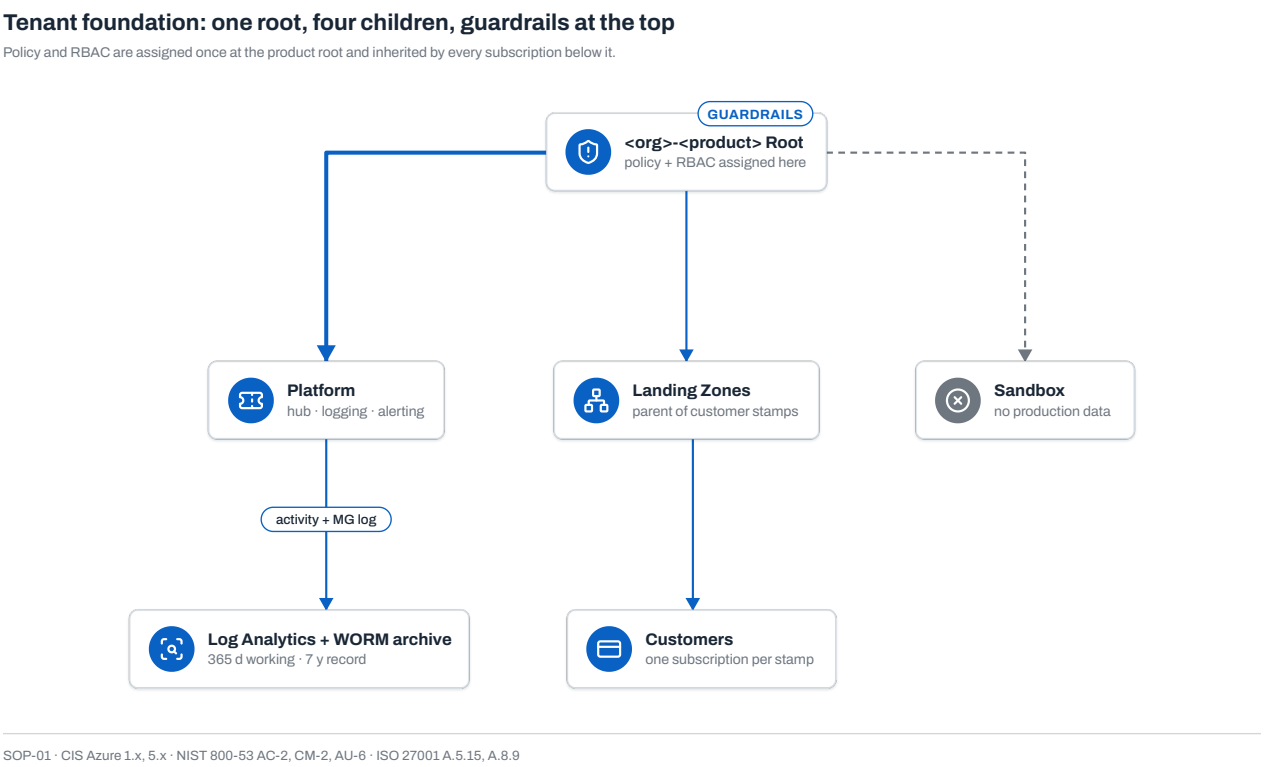


Figure 1. The product root carries the guardrails; everything below inherits them. Platform holds the shared services, Customers holds the stamps.

1.2 Guardrails: Deny, Modify and Audit

Effect	What it does	Used here for
Deny	Refuses the request at Azure Resource Manager. Nothing is created.	Allowed regions, no public IP on a NIC, required tags on resource groups
Modify	Changes the request on the way in (for example adds a tag). Needs a managed identity on the assignment.	Inherit tags from the resource group onto every resource
Audit	Lets the request through and records it as non-compliant.	CIS, NIST SP 800-53, ISO 27001, HIPAA and PCI initiatives, which produce evidence

CAUTION · A DENY LOOKS LIKE AN OUTAGE

A Deny policy refuses the request with a short error. Teams often read a tag refusal as a region block or a quota problem and ask for the wrong rule to be widened. Before changing any guardrail, read the refusing assignment's name from the activity log event (Chapter 5).

1.3 Logging: a working window and a record

Log Analytics keeps 365 days of searchable logs; that is the working window. A geo-redundant storage account with a time-based **WORM** (write once, read many) policy keeps the regulatory record for seven years. Changes made at a management group are written to that group's own activity log, which a subscription-level export never sees, so both are streamed.

CHAPTER 2

Access, Prerequisites and Planning

2.1 Level of access required

Grant the narrowest role that does the job, for the shortest time. Where a role is only needed for one step, activate it through Privileged Identity Management for that step and let it expire.

Task	Least-privilege role	Notes
Create management groups at the tenant root	Management Group Contributor at the tenant root, or Owner	Global Administrator elevation is not needed if the root grant exists
Move a subscription under Customers	Owner on the subscription and write on the target group	An ABAC condition on Owner can block it; ask for the unconditioned role
Create the five security groups	Groups Administrator (Entra ID)	Azure Owner does not include any directory role
Assign policy and roles at the root	Owner, or Resource Policy Contributor plus User Access Administrator	Policy identities need Tag Contributor for Modify
Lock the log archive	Data owner only	Locking is permanent; never done by the engineer

> **GOOD PRACTICE · SAY WHY A ROLE IS NEEDED**
>
> When asking a tenant administrator for access, name the exact step and role. "Owner on the subscription so the build can assign Tag Contributor to the policy identity" gets approved; "admin access" does not.

2.2 Prerequisites checklist

- [ ] Subscription IDs and tenant ID recorded in the parameter file (never in a public repository).
- [ ] Two allowed regions chosen, with the paired region for backup.
- [ ] Required tag names agreed (for example Owner, DataClassification, Client).
- [ ] Security mailbox for alerts agreed with the client.
- [ ] Az PowerShell 12 or later on PowerShell 7.4; `Test-…Access.ps1` ends READY.
- [ ] Written change window from the data owner.

2.3 Decisions to make up front

Decision	Guidance
One subscription or two	Two (platform and customer) is cleaner. One is acceptable for a first customer if written down; every per-subscription step then runs once.
Initiatives in Audit or Deny	Audit. Frameworks produce evidence; the small set of Deny rules holds the hard lines.
Immutability locked or unlocked	Unlocked at build. The data owner locks it after review with a separate script.
Naming standard	`<product>-<workload>-<env>-<region>-<type>-<nn>`; tenant-wide objects prefixed `<org>-<product>`.

CHAPTER 3

Procedure in the Azure Portal

Use this chapter to learn the build, review it, or rebuild one piece by hand in an emergency. For a real build run the template in Chapter 4; it does the same steps in the same order and leaves evidence behind.

> **NOTE · HOW THESE SCREENS WERE CAPTURED**
>
> Screens in this chapter were captured read-only from a live engagement tenant. Client, product and people names, tenant, subscription and object IDs, email addresses and public IPs are replaced (`org`, `app`, `Contoso`, `00000000-…`, `203.0.113.10`). Compliance figures and unrelated rows are blurred. Portal labels move over time; the setting names are what matter.

STEP 1 Create the management group tree

1. In the portal search bar type **Management groups**, then select `+ Create`.
2. Enter the ID `<org>-<product>` and a display name, and save. This is the product root.
3. Open the root and repeat for `-platform`, `-landingzones` and `-sandbox`; open Landing Zones and create `-customers`.
4. Open **Customers**, select `Add subscription` and choose the customer subscription. Use `Expand / Collapse all` to check the tree.

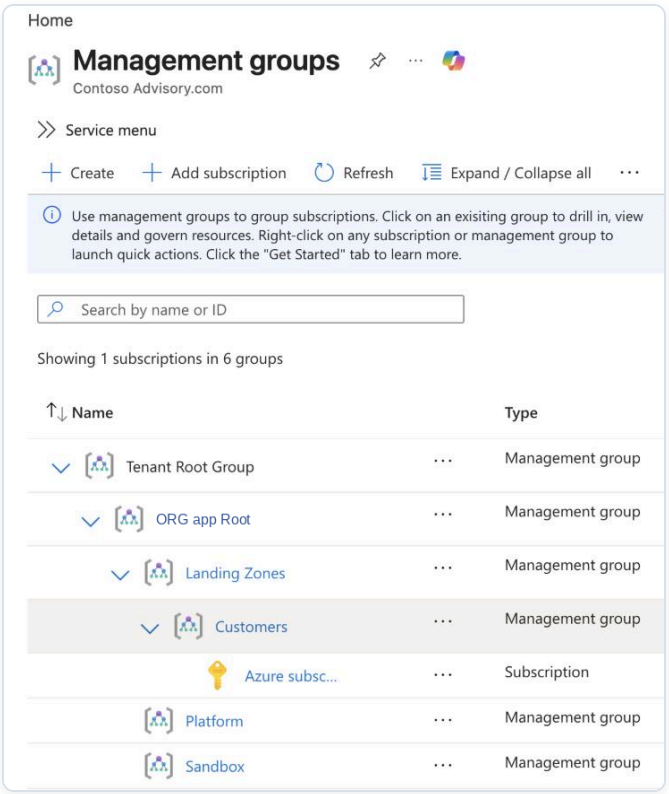


Figure 2. The finished tree: product root, Landing Zones › Customers holding the subscription, Platform and Sandbox.

STEP 2 Create the five security groups

1. Go to **Microsoft Entra ID › Groups › New group**. Group type **Security**, membership type **Assigned**.
2. Create `sg-az-<org>-<product>-owners`, `-platform-ops`, `-customer-ops`, `-security` and `-auditors`, each with an owner.
3. Leave membership to the client. The build never adds people to groups.

STEP 3 Assign the guardrail policies at the root

1. Open **Policy › Assignments** and select **Assign policy**. Under **Scope** choose the product root management group.
2. Assign one at a time: *Allowed locations* (your two regions), *Network interfaces should not have public IPs* and *Secure transfer to storage accounts should be enabled*.
3. For each required tag assign *Require a tag on resource groups* (Deny) and *Inherit a tag from the resource group if missing* (Modify). On the **Remediation** tab tick **Create a Managed Identity**.

4. Select **Assign initiative** for CIS Azure Foundations, NIST SP 800-53 R5, ISO 27001, HIPAA HITRUST and PCI DSS v4, all in Audit.

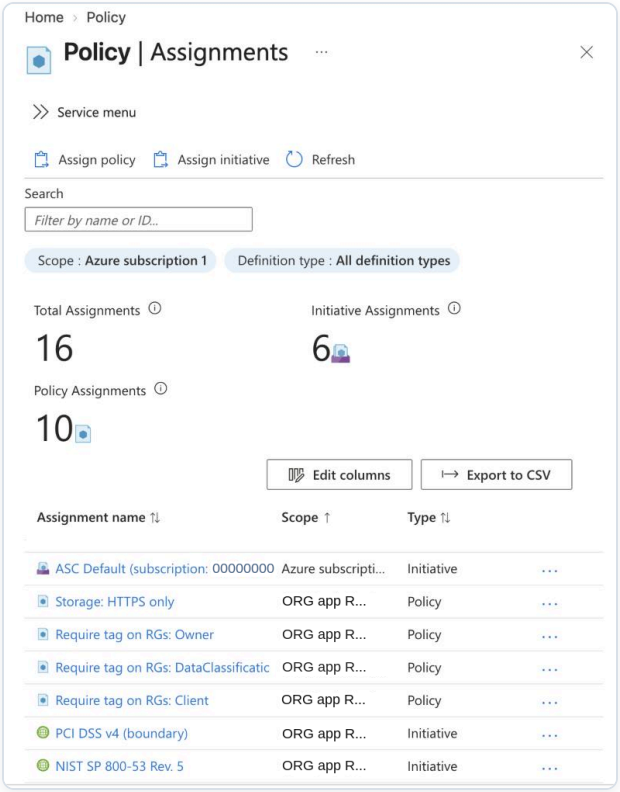


Figure 3. Assignments inherited from the product root. Deny rules hold the hard lines; initiatives produce evidence.

> **WHY IT MATTERS · DENY THE RESOURCE GROUP, NOT EVERY RESOURCE**
>
> Requiring tags on every resource also blocks the ones Azure creates for you (OS disks, private endpoint NICs). Deny on the resource group and inherit with Modify instead.

STEP 4 Build central logging

1. In the platform resource group create a **Log Analytics workspace**, pricing tier Pay-as-you-go, retention 365 days.
2. Create a **Storage account** for the archive: Geo-redundant, minimum TLS 1.2, public blob access off, public network access off.
3. In **Data storage › Containers** create the `am-*` containers and add a **time-based retention policy** of 2,557 days with protected append writes allowed.
4. Leave the policy **unlocked**. Locking is permanent and is the data owner's decision.

STEP 5 Stream the activity log

1. Open **Monitor › Activity log › Export Activity Logs** and select **+ Add diagnostic setting**.
2. Tick Administrative, Security, Policy, Alert and ServiceHealth; choose **Send to Log Analytics workspace**; **Save**.
3. For the product root management group, set the diagnostic setting with the template (step 5 of the code). The portal does not offer it.

STEP 6 Create the hub network and locks

1. Create the hub **Virtual network** with `snet-mgmt`, `snet-appgw` and the reserved `AzureFirewallSubnet` and `AzureBastionSubnet`.
2. On the platform and hub resource groups open **Settings › Locks**, select **+ Add**, lock type **Delete**.

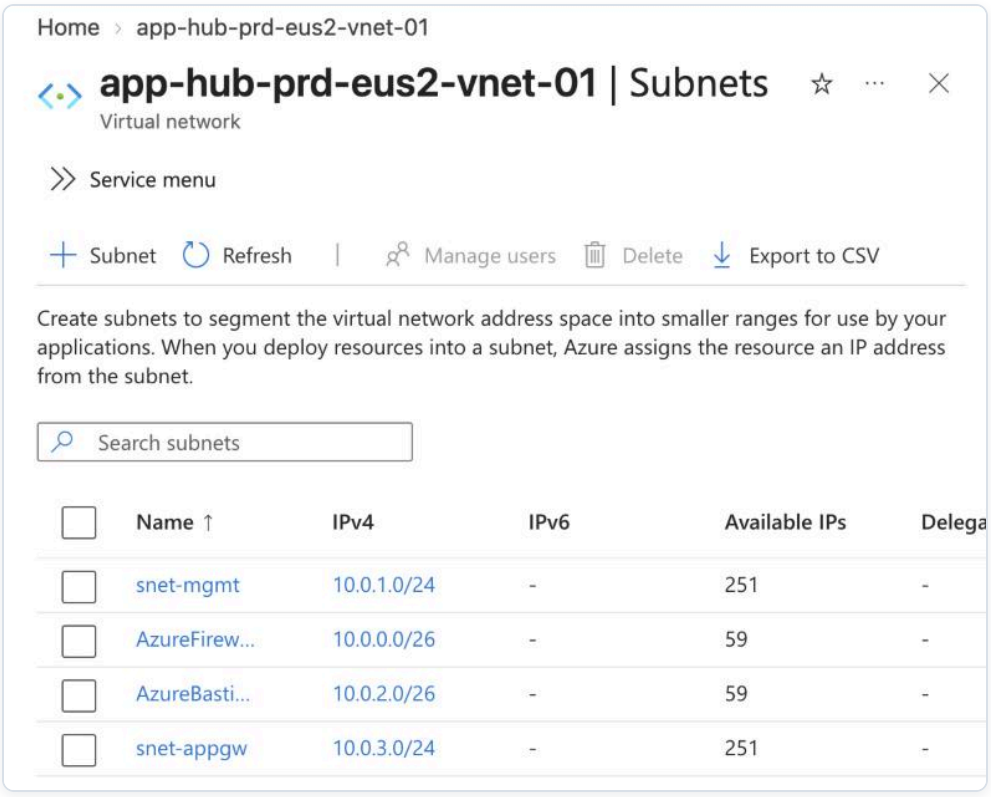


Figure 4. Hub subnets: management, gateway, and the two reserved names kept free for a firewall and Bastion.

CHAPTER 4

Template · Foundation as Code

The template is the version of record. It is idempotent: a rerun converges the tenant instead of duplicating resources, and every value specific to a client lives in the parameter file. Run it with `-WhatIf` first, then for real, and keep it in source control beside the evidence it produced.

SOP-01-LANDING-ZONE.PS1 · FOUNDATION · BASE TEMPLATE — POWERSHELL

```powershell
<#  SOP-01 template: tenant foundation (management groups, RBAC groups, policy, logging)
    Sanitized pattern. Replace <...> values in tenant.parameters.json. Run with -WhatIf first.
    Needs: Az 12+, Owner on the subscription, management group write at the tenant root,
    Groups Administrator (Entra) to create the security groups.                          #>
[CmdletBinding(SupportsShouldProcess)]
param([Parameter(Mandatory)][string]$ParameterFile)
$ErrorActionPreference = 'Stop'
$cfg     = Get-Content $ParameterFile -Raw | ConvertFrom-Json
$OrgProd = "$($cfg.orgPrefix)-$($cfg.product)".ToLower()
$Tags    = @{}; $cfg.tags.PSObject.Properties | ForEach-Object { $Tags[$_.Name] = $_.Value }
$root    = "/providers/Microsoft.Management/managementGroups/$OrgProd"

# Naming standard: <product>-<workload>-<env>-<region>-<type>-<nn>
function New-ResName([string]$Workload, [string]$Type) {
    ('{0}-{1}-{2}-eus2-{3}-01' -f $cfg.product, $Workload, $cfg.environment, $Type).ToLower()
}

# 1. Management group hierarchy: Root > Platform | Landing Zones > Customers | Sandbox
$plan = @(
  @{ Name = $OrgProd;                Display = "$OrgProd Root";  Parent =
"/providers/Microsoft.Management/managementGroups/$($cfg.tenantId)" }
  @{ Name = "$OrgProd-platform";     Display = 'Platform';       Parent = $root }
  @{ Name = "$OrgProd-landingzones"; Display = 'Landing Zones';  Parent = $root }
  @{ Name = "$OrgProd-customers";    Display = 'Customers';      Parent =
"/providers/Microsoft.Management/managementGroups/$OrgProd-landingzones" }
  @{ Name = "$OrgProd-sandbox";      Display = 'Sandbox';        Parent = $root }
)
foreach ($mg in $plan) {
  if (-not (Get-AzManagementGroup -GroupName $mg.Name -ErrorAction SilentlyContinue) -and
$PSCmdlet.ShouldProcess($mg.Name, 'Create MG')) {
    New-AzManagementGroup -GroupName $mg.Name -DisplayName $mg.Display -ParentId $mg.Parent | Out-Null
  }
}
New-AzManagementGroupSubscription -GroupName "$OrgProd-customers" -SubscriptionId $cfg.customerSubscriptionId -
WhatIf:$WhatIfPreference

# 2. RBAC: groups, never users. Privileged roles are PIM-eligible (Entra ID P2), not standing.
$groups = 'owners','platform-ops','customer-ops','security','auditors' | ForEach-Object {
  $n = "sg-az-$OrgProd-$_"
  $g = Get-AzADGroup -DisplayName $n -ErrorAction SilentlyContinue
  if (-not $g) { $g = New-AzADGroup -DisplayName $n -MailNickname ($n -replace '[^a-zA-Z0-9]','') -
SecurityEnabled }
  @{ $_ = $g }
}
$G = @{}; $groups | ForEach-Object { $G += $_ }
New-AzRoleAssignment -ObjectId $G['security'].Id     -RoleDefinitionName 'Security Reader' -Scope $root
New-AzRoleAssignment -ObjectId $G['auditors'].Id     -RoleDefinitionName 'Reader'          -Scope $root
New-AzRoleAssignment -ObjectId $G['platform-ops'].Id -RoleDefinitionName 'Contributor'     -Scope "$root-
platform"
# Owner (root) and Contributor (Customers) are assigned as PIM *eligible*; see SOP-04.
```

```
# 3. Central logging: Log Analytics (working window) + GRS archive with WORM (regulatory record)
$rgPlat = New-ResName plat rg
New-AzResourceGroup -Name $rgPlat -Location $cfg.location -Tag $Tags | Out-Null
$law = New-AzOperationalInsightsWorkspace -ResourceGroupName $rgPlat -Name (New-ResName plat law) `
         -Location $cfg.location -Sku PerGB2018 -RetentionInDays $cfg.logRetentionDays
$sa  = New-AzStorageAccount -ResourceGroupName $rgPlat -Name ((New-ResName plat st) -replace '-','') -Location
$cfg.location `
         -SkuName Standard_GRS -Kind StorageV2 -MinimumTlsVersion TLS1_2 -AllowBlobPublicAccess $false -
EnableHttpsTrafficOnly $true
foreach ($t in 'azureactivity','azurediagnostics','syslog') {
  New-AzRmStorageContainer -ResourceGroupName $rgPlat -StorageAccountName $sa.StorageAccountName -Name "am-$t" |
Out-Null
  # Unlocked time-based WORM; the data owner locks it later (locking is irreversible).
  Set-AzRmStorageContainerImmutabilityPolicy -ResourceGroupName $rgPlat -StorageAccountName
$sa.StorageAccountName `
     -ContainerName "am-$t" -ImmutabilityPeriod $cfg.immutableRetentionDays -AllowProtectedAppendWrite $true |
Out-Null
}

# 4. Azure Policy at the product root: hard lines Deny, frameworks Audit
function Builtin($like) { Get-AzPolicyDefinition -Builtin | Where-Object { $_.DisplayName -like $like } | Select-
Object -First 1 }
New-AzPolicyAssignment -Name 'allowed-locations'  -Scope $root -PolicyDefinition (Builtin 'Allowed locations') `
  -PolicyParameterObject @{ listOfAllowedLocations = @($cfg.allowedLocations) }
New-AzPolicyAssignment -Name 'deny-nic-public-ip' -Scope $root -PolicyDefinition (Builtin 'Network interfaces
should not have public IPs')
foreach ($tag in $cfg.requiredTags) {
  New-AzPolicyAssignment -Name "rgtag-$($tag.ToLower())"  -Scope $root -PolicyDefinition (Builtin 'Require a tag
on resource groups') -PolicyParameterObject @{ tagName = $tag }
  New-AzPolicyAssignment -Name "inhtag-$($tag.ToLower())" -Scope $root -PolicyDefinition (Builtin 'Inherit a tag
from the resource group if missing') `
    -PolicyParameterObject @{ tagName = $tag } -IdentityType SystemAssigned -Location $cfg.location
}
$cis = Get-AzPolicySetDefinition -Builtin | Where-Object DisplayName -like 'CIS Microsoft Azure Foundations
Benchmark v2*' | Select-Object -First 1
New-AzPolicyAssignment -Name 'cis-azure-foundations' -Scope $root -PolicySetDefinition $cis -IdentityType
SystemAssigned -Location $cfg.location

# 5. Activity log of the subscription AND of the root management group -> workspace
$diag = @{ properties = @{ workspaceId = $law.ResourceId; logs = @(@{category='Administrative';enabled=$true},
@{category='Policy';enabled=$true}, @{category='Security';enabled=$true}) } } | ConvertTo-Json -Depth 6
Invoke-AzRestMethod -Method PUT -Path
"/subscriptions/$($cfg.customerSubscriptionId)/providers/Microsoft.Insights/diagnosticSettings/activity-to-law?
api-version=2021-05-01-preview" -Payload $diag
Invoke-AzRestMethod -Method PUT -Path "$root/providers/Microsoft.Insights/diagnosticSettings/mg-activity-to-law?
api-version=2020-01-01-preview" -Payload $diag

# 6. Hub network and delete locks
$rgHub = New-ResName hub rg
New-AzResourceGroup -Name $rgHub -Location $cfg.location -Tag $Tags | Out-Null
$sn = @(
  New-AzVirtualNetworkSubnetConfig -Name 'snet-mgmt'  -AddressPrefix $cfg.hub.managementSubnet
  New-AzVirtualNetworkSubnetConfig -Name 'snet-appgw' -AddressPrefix $cfg.hub.applicationGatewaySubnet
)
New-AzVirtualNetwork -ResourceGroupName $rgHub -Name (New-ResName hub vnet) -Location $cfg.location -
AddressPrefix $cfg.hub.addressSpace -Subnet $sn | Out-Null
foreach ($rg in $rgPlat, $rgHub) { New-AzResourceLock -LockName 'do-not-delete' -LockLevel CanNotDelete -
ResourceGroupName $rg -Force | Out-Null }

# 7. Exit criterion: policy compliance at 100 percent
Start-AzPolicyComplianceScan -AsJob | Out-Null
Get-AzPolicyStateSummary -ManagementGroupName $OrgProd | Select-Object -ExpandProperty Results
```

TENANT.PARAMETERS.TEMPLATE.JSON · PARAMETER FILE · PLACEHOLDERS — JSON

```json
{
  "_comment": "Template. Replace every <...> value. Never commit real tenant or subscription IDs to a public repo.",
  "orgPrefix": "<org>",
  "product": "<product>",
  "tenantId": "<tenant-id>",
  "platformSubscriptionId": "<platform-subscription-id>",
  "customerSubscriptionId": "<customer-subscription-id>",
  "location": "eastus2",
  "pairedLocation": "centralus",
  "allowedLocations": ["eastus2", "centralus"],
  "environment": "prd",
  "customerCode": "c001",
  "tags": {
    "Client": "<client-name>",
    "Owner": "<owner-mailbox>",
    "Environment": "prd",
    "DataClassification": "Confidential",
    "ControlBaseline": "CIS-Azure-2.x;NIST-800-53-R5;ISO-27001;HIPAA"
  },
  "requiredTags": ["Owner", "DataClassification", "Client"],
  "alertEmails": ["<security-mailbox>"],
  "logRetentionDays": 365,
  "immutableRetentionDays": 2557,
  "hub":   { "addressSpace": "10.0.0.0/16", "managementSubnet": "10.0.1.0/24", "applicationGatewaySubnet": "10.0.3.0/24" },
  "stamp": { "addressSpace": "10.1.0.0/16", "appSubnet": "10.1.1.0/24", "privateEndpointSubnet": "10.1.2.0/24",
             "vmSize": "Standard_D2s_v6", "appPort": 3001, "backupRetentionDays": 35 }
}
```

EXAMPLE · RUNNING THE TEMPLATE — POWERSHELL

```powershell
Connect-AzAccount -TenantId <tenant-id>
./SOP-01-landing-zone.ps1 -ParameterFile ./tenant.parameters.json -WhatIf   # list every change
./SOP-01-landing-zone.ps1 -ParameterFile ./tenant.parameters.json           # build
Get-AzPolicyStateSummary -ManagementGroupName <org>-<product>                 # exit check
```

NOTE · SINGLE SUBSCRIPTION

When the platform and customer subscription IDs are the same, every per-subscription step runs once. Record that choice in the design document; it is a scope decision, not a shortcut.

CHAPTER 5

Verification and Troubleshooting

5.1 Verify the build

- [ ] `Get-AzManagementGroup -GroupName <org>-<product> -Expand -Recurse` shows the five groups and the subscription under Customers.
- [ ] Role assignments at the root are held by groups only (export and hash).
- [ ] **Policy › Compliance** at the root: Deny assignments at 100 percent; initiatives exported to CSV.
- [ ] Archive containers show a time-based retention policy, unlocked.
- [ ] Evidence manifest records run ID, operator and UTC time (ALCOA+).

5.2 Troubleshooting

Symptom	Likely cause and fix
Subscription move refused	Owner carries an ABAC condition or the target group denies writes. Ask for an unconditioned Owner, then rerun; the script warns and continues.
Group creation fails	The account has no Entra directory role. Ask for Groups Administrator for the build, or have the five groups created by name.
Deployment refused with RequestDisallowedByPolicy	Read the `policies` property of the activity log event. Most often a required tag is missing on a new resource group.
Modify policy never adds tags	The assignment identity lacks Tag Contributor at the root, or no remediation task was run for existing resources.
No management group events in the workspace	The root group diagnostic setting is missing. Set it with the template.

5.3 Rollback

Remove resources in reverse order: locks first, then resource groups, then policy assignments, then role assignments, then management groups (move the subscription back to the tenant root first). Never delete the archive storage account; an unlocked WORM policy still blocks deleting the data inside it until the retention ends.

CHAPTER 6

Summary, Glossary and Review

6.1 Chapter summary

- Guardrails, roles and logging are set once at the product root and inherited by everything below.
- Deny holds the hard lines; Modify fixes tags quietly; Audit produces framework evidence.
- Activity logs from the subscription and the root management group both reach the workspace; the WORM archive is the record.
- The template is the version of record; the portal is for learning, review and emergencies.

Check your understanding

1. Why are tags required on resource groups rather than on every resource?
2. Which log never reaches a subscription-level activity log export, and how is it captured?
3. Who locks the immutability policy on the archive, and why not the engineer?
4. A deployment in an allowed region is refused. What do you read first?
5. What does the exit criterion of this SOP say, in one sentence?

6.2 Glossary

Management group	Container for subscriptions; policy and RBAC assigned here are inherited.
Initiative	A set of policy definitions assigned together, for example a framework benchmark.
Modify effect	Policy effect that changes a request, such as adding a tag; needs a managed identity.
WORM	Write once, read many: data cannot be changed or deleted for the retention period.
Idempotent	Safe to run again: a rerun converges to the same state without duplicates.

6.3 Framework references

Framework	Controls
CIS Azure Foundations	1.x, 5.x

Framework	Controls
NIST CSF 2.0	GV.PO, PR.AA, DE.CM
NIST SP 800-53 R5	AC-2, CM-2, CM-7, AU-6, AU-9
ISO/IEC 27001:2022	A.5.15, A.8.9, A.8.15